

Machine Learning - Exploratory Data Analysis – Data Preprocessing

Agenda

- ❑ Machine Learning – Data preprocessing
- ❑ Machine Learning – EDA plots

Machine Learning – Data Preprocessing

What is Data preprocessing?

- Data scientists usually come across many datasets/raw data which may contain noise and not fit enough to be readily processed by machine learning algorithms
- Data scientist needs to preprocess the raw data before it is fed into various machine learning algorithms
- **Data Preprocessing** is the process of preparing the data for analysis.

Machine Learning – Data Preprocessing

Data preprocessing - Techniques

- Data preprocessing is an important part of model building using Machine learning
- Data preprocessing takes major chunk of model building effort depending on dataset.
- Number of Statistical, mathematical, computational and graphical techniques available to preprocess the Dataset and make it ready for ML algorithm.
- These techniques are not limited and may not be performed in order. Some datasets require lot of preprocessing and some require very minimal preprocessing.

Machine Learning – Data Preprocessing

Data preprocessing - Techniques

Some commonly used Data preprocessing techniques are..

- ❖ Data loading(into data frames)
- ❖ Handling missing data
- ❖ Handling outliers
- ❖ Scaling
- ❖ Normalization
- ❖ Binarization
- ❖ Standardization
- ❖ Label Encoding
- ❖ One Hot Encoding
- ❖ Data Visualization
- ❖ Feature Engineering
- ❖ Dimensionality Reduction

Machine Learning – Data Preprocessing

Data preprocessing - Techniques

- Lets work on an Example dataset

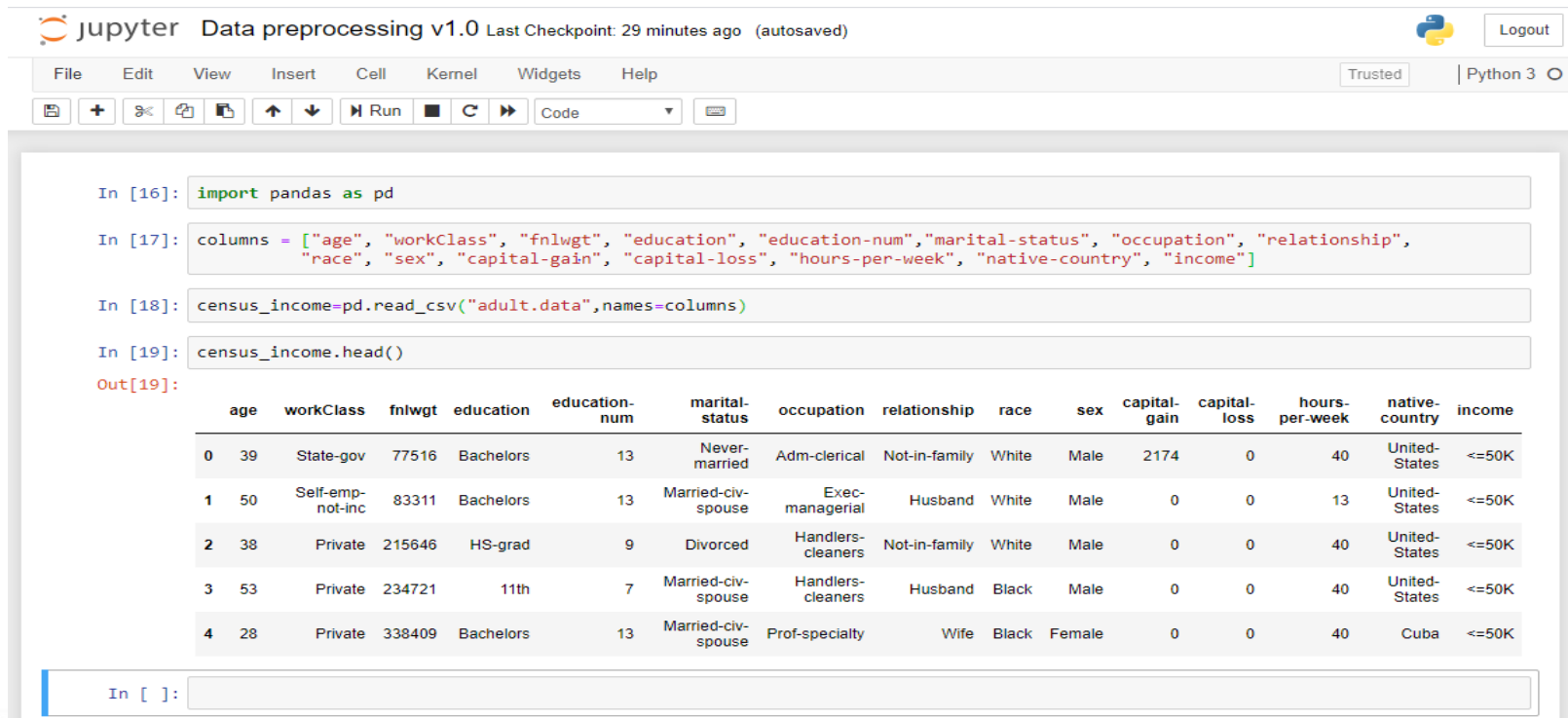
Dataset: Census income data

- Download dataset from UCI Machine Learning Repository
- Link: <https://archive.ics.uci.edu/ml/datasets/Adult>
- This data was extracted from the census bureau database.

Machine Learning – Data Preprocessing

Data preprocessing - Data Loading

- Read dataset into dataframe(pandas)



The image shows a Jupyter Notebook interface with the title "Data preprocessing v1.0". The notebook contains four code cells. The first cell imports pandas as pd. The second cell defines a list of column names. The third cell reads a CSV file named "adult.data" into a dataframe named "census_income". The fourth cell displays the first five rows of the dataframe using the head() method. The output shows a table with 15 columns: age, workClass, fnlwtg, education, education-num, marital-status, occupation, relationship, race, sex, capital-gain, capital-loss, hours-per-week, native-country, and income. The first five rows of data are displayed.

```
In [16]: import pandas as pd

In [17]: columns = ["age", "workClass", "fnlwtg", "education", "education-num", "marital-status", "occupation", "relationship", "race", "sex", "capital-gain", "capital-loss", "hours-per-week", "native-country", "income"]

In [18]: census_income=pd.read_csv("adult.data",names=columns)

In [19]: census_income.head()
```

Out[19]:

	age	workClass	fnlwtg	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	native-country	income
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K

Machine Learning – Data Preprocessing

Data preprocessing - Handling missing data

- Missing data in a dataset can be a blank or NaN or special symbols representing missing data.
- A dataset has missing value because of number of reasons
 - ❖ not recorded during creation
 - ❖ Optional entry
 - ❖ Data corruption
- Handling missing data is important as most of the machine learning algorithms do not support data with missing values.

Machine Learning – Data Preprocessing

Data preprocessing - Handling missing data

There are number of techniques available to handle missing data

- **Deleting Missing data Rows:**

- ❖ Delete the row if it is having lot of missing values in the columns.
- ❖ It is a fail safe method advisable if the dataset is large enough.
- ❖ Not advisable if data is less and lot of missing data records.

- **Replace with statistical values:**

- ❖ Replace missing values with Mean/Median/Mode of corresponding attribute distribution.
- ❖ This process also called data imputation
- ❖ Better approach if data set is small, prevents data loss
- ❖ Imputed value may add bias or variance

Machine Learning – Data Preprocessing

Data preprocessing - Handling missing data

There are number of techniques available to handle missing data

- **Missing categorical values:**
 - ❖ Replacing with statistical values works only for numerical data
 - ❖ For categorical missing values, data imputation can be done by creating a unique category for missing data or replace with most occurred category.
 - ❖ If there is lot of missing data, better exclude variable from model building
- **Predicting missing values:**
 - ❖ Missing values can be predicted using existing algorithms(libraries)
 - ❖ Can also be predicted from by building model with rest of the data of same dataset

Machine Learning – Data Preprocessing

Data preprocessing - Handling missing data

Example:

```
import pandas as pd
Import numpy as np
dict1 = {
    'Sub1':[95,93,94,85,np.nan],
    'Sub2':[85, np.nan,90 ,75,np.nan],
    'Sub3': [30, 45, 55, np.nan,np.nan],
    'Sub4':[np.nan, 40, 80, 97,np.nan],
    'Sub5':[np.nan,np.nan,np.nan,np.nan,np.nan],
    'Sub6':[75,77,73,np.nan,np.nan]}
data= pd.DataFrame(dict1)
```

Machine Learning – Data Preprocessing

Data preprocessing - Handling missing data

Example:

```
import pandas as pd
Import numpy as np
dict1 = {
    'Sub1':[95,93,94,85,np.nan],
    'Sub2':[85, np.nan,90 ,75,np.nan],
    'Sub3': [30, 45, 55, np.nan,np.nan],
    'Sub4':[np.nan, 40, 80, 97,np.nan],
    'Sub5':[np.nan,np.nan,np.nan,np.nan,np.nan],
    'Sub6':[75,77,73,np.nan,np.nan]}
data= pd.DataFrame(dict1)
```

Machine Learning – Data Preprocessing

Data preprocessing - Handling missing data

Example:

- Checking for null values

```
import pandas as pd  
data.isnull()
```

- Dropping record having all null values
`data.dropna(how = 'all')`

```
In [53]: data.dropna(how = 'all')
```

```
Out[53]:
```

	Sub1	Sub2	Sub3	Sub4	Sub5	Sub6
0	95.0	85.0	30.0	NaN	NaN	75.0
1	93.0	NaN	45.0	40.0	NaN	77.0
2	94.0	90.0	55.0	80.0	NaN	73.0
3	85.0	75.0	NaN	97.0	NaN	NaN

Machine Learning – Data Preprocessing

Data preprocessing - Handling missing data

Example:

- Fill missing values with 0
`data.fillna(0)`

```
In [54]: data.fillna(0)
```

```
Out[54]:
```

	Sub1	Sub2	Sub3	Sub4	Sub5	Sub6
0	95.0	85.0	30.0	0.0	0.0	75.0
1	93.0	0.0	45.0	40.0	0.0	77.0
2	94.0	90.0	55.0	80.0	0.0	73.0
3	85.0	75.0	0.0	97.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0

- Replace missing values with -1
`data.replace(to_replace = np.nan, value = -1)`

```
In [56]: data.replace(to_replace = np.nan, value = -1) |
```

```
Out[56]:
```

	Sub1	Sub2	Sub3	Sub4	Sub5	Sub6
0	95.0	85.0	30.0	-1.0	-1.0	75.0
1	93.0	-1.0	45.0	40.0	-1.0	77.0
2	94.0	90.0	55.0	80.0	-1.0	73.0
3	85.0	75.0	-1.0	97.0	-1.0	-1.0
4	-1.0	-1.0	-1.0	-1.0	-1.0	-1.0

Machine Learning – Data Preprocessing

Data preprocessing - Handling missing data

Example:

- Fill missing values with columns mean
`data.fillna(data.mean())`

```
In [61]: data.fillna(data.mean())
```

```
Out[61]:
```

	Sub1	Sub2	Sub3	Sub4	Sub5	Sub6
0	95.00	85.000000	30.000000	72.333333	NaN	75.0
1	93.00	83.333333	45.000000	40.000000	NaN	77.0
2	94.00	90.000000	55.000000	80.000000	NaN	73.0
3	85.00	75.000000	43.333333	97.000000	NaN	75.0
4	91.75	83.333333	43.333333	72.333333	NaN	75.0

- Fill missing values with imputation function
`data.interpolate(method='linear', limit_direction='forward')`

```
In [58]: data.interpolate(method='linear', limit_direction='forward')
```

```
Out[58]:
```

	Sub1	Sub2	Sub3	Sub4	Sub5	Sub6
0	95.0	85.0	30.0	NaN	NaN	75.0
1	93.0	87.5	45.0	40.0	NaN	77.0
2	94.0	90.0	55.0	80.0	NaN	73.0
3	85.0	75.0	55.0	97.0	NaN	73.0
4	85.0	75.0	55.0	97.0	NaN	73.0

Machine Learning – Data Preprocessing

Data preprocessing - Handling missing data

Example: Census income dataset

- Census income dataset doesn't have any missing values.

```
In [7]: census_income.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 15 columns):
age                32561 non-null int64
workClass          32561 non-null object
fnlwgt            32561 non-null int64
education          32561 non-null object
education-num      32561 non-null int64
marital-status     32561 non-null object
occupation         32561 non-null object
relationship       32561 non-null object
race              32561 non-null object
sex               32561 non-null object
capital-gain       32561 non-null int64
capital-loss       32561 non-null int64
hours-per-week     32561 non-null int64
native-country     32561 non-null object
income            32561 non-null object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
```

Machine Learning – Data Preprocessing

Data preprocessing - Handling Outliers

- An outlier is an observation in a dataset that is different or far away the other observations.
- An outlier can happen because of an error scenario or a genuine scenario.
- There are number of ways to idenity the outliers
 - ❖ Box plot of distribution
 - ❖ scatter plot
 - ❖ Z-score
 - ❖ IQR

Machine Learning – Data Preprocessing

Data preprocessing - Handling Outliers

Box Plot:

- In descriptive statistics, a box plot is a method for graphically depicting groups of numerical data through their quartiles

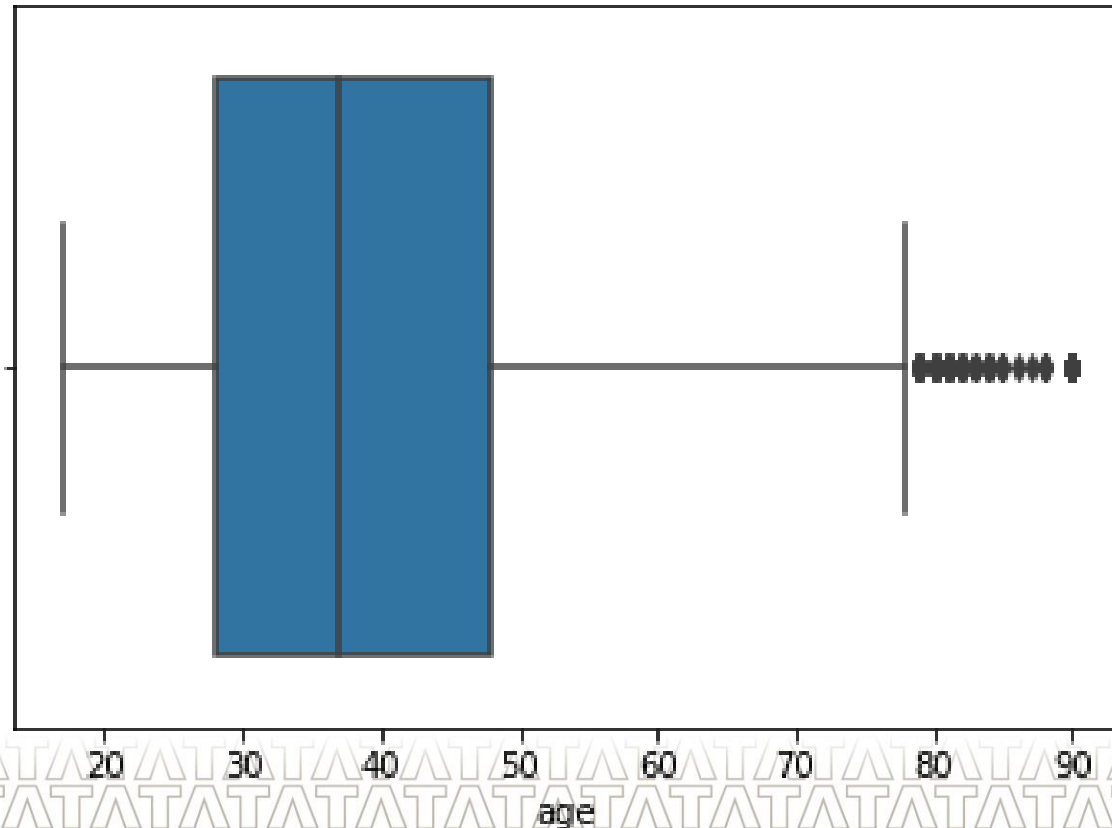
```
import seaborn as sns  
sns.boxplot(x=census_income['age'])
```

- Points which are falling outside the box i.e between 80-90 years are outliers

Machine Learning – Data Preprocessing

Data preprocessing - Handling Outliers

Box Plot:



Machine Learning – Data Preprocessing

Data preprocessing - Handling Outliers

Scatter Plot:

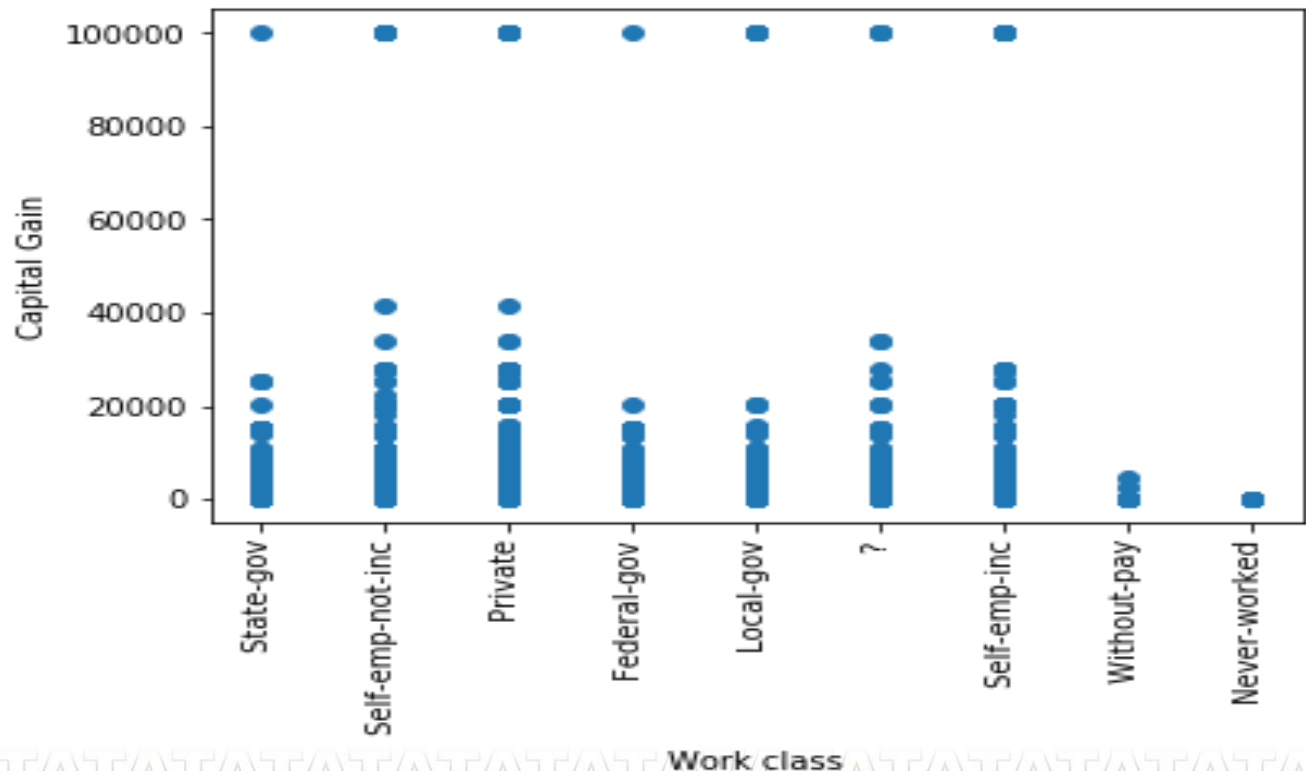
- Scatter plot gives relation between two variables

```
import matplotlib.pyplot as plt
plt.scatter(census_income['workClass'],
            census_income['capital-gain'])
plt.xlabel('Work class')
plt.xticks(rotation=90)
plt.ylabel('Capital Gain')
plt.show()
```

Machine Learning – Data Preprocessing

Data preprocessing - Handling Outliers

Scatter Plot:



Machine Learning – Data Preprocessing

Data preprocessing - Handling Outliers

Z-score:

- The Z-score is to describe a data point by finding their relationship with the Standard Deviation and Mean of the group of data points.
- In most of the cases if the Z-score value is greater than 3 or less than -3, then data point will be identified as outliers.

```
from scipy import stats
import numpy as np
z_score=np.abs(stats.zscore(census_income['age']))
print(census_income.iloc[np.where(z_score > 3)].head())
```

Machine Learning – Data Preprocessing

Data preprocessing - Handling Outliers

Z-score:

	age	workClass	fnlwgt	education	education-num	\
222	90	Private	51744	HS-grad	9	
430	80	?	107762	HS-grad	9	
918	81	Self-emp-not-inc	136063	HS-grad	9	
1040	90	Private	137018	HS-grad	9	
1168	88	Self-emp-not-inc	206291	Prof-school	15	

	marital-status	occupation	relationship	race	sex	\
222	Never-married	Other-service	Not-in-family	Black	Male	
430	Widowed	?	Not-in-family	White	Male	
918	Married-civ-spouse	Exec-managerial	Husband	White	Male	
1040	Never-married	Other-service	Not-in-family	White	Female	
1168	Married-civ-spouse	Prof-specialty	Husband	White	Male	

	capital-gain	capital-loss	hours-per-week	native-country	income
222	0	2206	40	United-States	<=50K
430	0	0	24	United-States	<=50K
918	0	0	30	United-States	<=50K
1040	0	0	40	United-States	<=50K
1168	0	0	40	United-States	<=50K

Machine Learning – Data Preprocessing

Data preprocessing - Handling Outliers

IQR:

- The interquartile range (IQR), is a measure of statistical dispersion, being equal to the difference between 75th and 25th percentiles, or between upper and lower quartiles, $IQR = Q3 - Q1$.
- Box plot use the IQR method to display data and outliers pictorially

```
Q1 = census_income['age'].quantile(0.25)
```

```
Q3 = census_income['age'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
print(IQR)
```

```
print(census_income[(census_income['age'] < (Q1 - 1.5 * IQR)) |  
(census_income['age'] > (Q3 + 1.5 * IQR))].head())
```

Machine Learning – Data Preprocessing

Data preprocessing - Handling Outliers

IQR:

```
In [82]: print(census_income[(census_income['age'] < (Q1 - 1.5 * IQR)) | (census_income['age'] > (Q3 + 1.5 * IQR))].head())
```

	age	workClass	fnlwgt	education	education-num	\
74	79	Private	124744	Some-college	10	
222	90	Private	51744	HS-grad	9	
430	80	?	107762	HS-grad	9	
918	81	Self-emp-not-inc	136063	HS-grad	9	
1040	90	Private	137018	HS-grad	9	

	marital-status	occupation	relationship	race	sex	\
74	Married-civ-spouse	Prof-specialty	Other-relative	White	Male	
222	Never-married	Other-service	Not-in-family	Black	Male	
430	Widowed	?	Not-in-family	White	Male	
918	Married-civ-spouse	Exec-managerial	Husband	White	Male	
1040	Never-married	Other-service	Not-in-family	White	Female	

	capital-gain	capital-loss	hours-per-week	native-country	income
74	0	0	20	United-States	<=50K
222	0	2206	40	United-States	<=50K
430	0	0	24	United-States	<=50K
918	0	0	30	United-States	<=50K
1040	0	0	40	United-States	<=50K

Machine Learning – Data Preprocessing

Data preprocessing - Handling Outliers

- The identified outliers can be corrected or removed or treated separately, but the decision depends upon the scenario.

Example:

- Outlier due to incorrect data entry – Remove, Correct
- Outlier due to exceptional cases like students with high IQ, High income at younger age – Keep / Handle

Machine Learning – Data Preprocessing

Data preprocessing - Scaling - Normalization

- The values of some features in a data set can vary between random values, which may cause issues while model building. These values can be scaled to given range

```
import numpy as np
from sklearn import preprocessing
input_data = np.array([[3, -1.5, 3, -6.4], [0, 3, -1.3, 4.1], [1, 2.3, -2.9, -4.3]])
ata_scaler = preprocessing.MinMaxScaler(feature_range = (0, 1))
data_scaled = ata_scaler.fit_transform(input_data)
print ("\nMin max scaled data = ", data_scaled)
data_normalized = preprocessing.normalize(input_data, norm = 'l1')
print ("\nL1 normalized data = ", data_normalized)
```

Machine Learning – Data Preprocessing

Data preprocessing - Standardization

- Standardization is a useful technique to transform attributes with a Gaussian distribution and differing means and standard deviations to a standard Gaussian distribution with a mean of 0 and a standard deviation of 1.

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler().fit(input_data)  
scaler.transform(input_data)
```

Machine Learning – Data Preprocessing

Data preprocessing - Further formatting

- Check for discrepancies in dataset

```
census_income['age'].unique()
```

```
array([39, 50, 38, 53, 28, 37, 49, 52, 31, 42, 30, 23, 32, 40, 34, 25, 43,  
      54, 35, 59, 56, 19, 20, 45, 22, 48, 21, 24, 57, 44, 41, 29, 18, 47,  
      46, 36, 79, 27, 67, 33, 76, 17, 55, 61, 70, 64, 71, 68, 66, 51, 58,  
      26, 60, 90, 75, 65, 77, 62, 63, 80, 72, 74, 69, 73, 81, 78, 88, 82,  
      83, 84, 85, 86, 87], dtype=int64)
```

```
census_income['workClass'].unique()
```

```
array([' State-gov', ' Self-emp-not-inc', ' Private', ' Federal-gov',  
      ' Local-gov', ' ?', ' Self-emp-inc', ' Without-pay',  
      ' Never-worked'], dtype=object)
```

Machine Learning – Data Preprocessing

Data preprocessing - Further formatting

- Below features having '?'
 - ❖ workClass
 - ❖ occupation
 - ❖ native-country
- Drop the above rows from dataset

```
census_income=census_income[census_income['workClass'] != '?']  
census_income=census_income[census_income['occupation'] != '?']  
census_income[census_income['native-country'] != '?']
```

Machine Learning – Data Preprocessing

Data preprocessing - Label Encoding

- Label encoding converts each values in a column into number from sklearn.preprocessing import LabelEncoder

```
census_income_2=census_income.copy()
```

```
In [151]: census_income_2.head()
```

```
Out[151]:
```

	age	workClass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	native-country	income
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K

Machine Learning – Data Preprocessing

Data preprocessing - Label Encoding

```
census_income_2=census_income.copy()
categorical_features = list(census_income_2.select_dtypes(include=['object']).columns)
label_encoder_feat = {}
for i, feature in enumerate(categorical_features):
    label_encoder_feat[feature] = LabelEncoder()
    census_income_2[feature]
    = label_encoder_feat[feature].fit_transform(census_income_2[feature])

census_income_2.head()
```

	age	workClass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	native-country	income
0	39	5	77516	9	13	4	0	1	4	1	2174	0	40	38	0
1	50	4	83311	9	13	2	3	0	4	1	0	0	13	38	0
2	38	2	215646	11	9	0	5	1	4	1	0	0	40	38	0
3	53	2	234721	1	7	2	5	0	2	1	0	0	40	38	0
4	28	2	338409	9	13	2	9	5	2	0	0	0	40	4	0

Machine Learning – Data Preprocessing

Data preprocessing - OneHot Encoding

- Label encoding gives number to categorical variables, this might confuse our model into thinking that a column has data with some kind of order or hierarchy when we clearly don't have it.
- OneHot encoding solves this issues
- OneHot splits the column into multiple binary (0's and 1's) columns depending on the number of categories present in that column.

Machine Learning – Data Preprocessing

Data preprocessing - OneHot Encoding

```
from sklearn.preprocessing import OneHotEncoder
census_income_3=census_income.copy()
categorical_features = list(census_income_3.select_dtypes(include=['object']).columns)
for feature in categorical_features:
    dummies = pd.get_dummies(census_income_3[feature], prefix=feature, drop_first=False)
    census_income_3 = pd.concat([census_income_3, dummies], axis=1)
    census_income_3 = census_income_3.drop([feature], axis=1)
census_income_3.head()
```

Machine Learning – Data Preprocessing

Data preprocessing - OneHot Encoding

```
In [180]: for feature in categorical_features:
          dummies = pd.get_dummies(census_income_3[feature], prefix=feature, drop_first=False)
          census_income_3 = pd.concat([census_income_3, dummies], axis=1)
          census_income_3 = census_income_3.drop([feature], axis=1)
          census_income_3.head()
```

Out[180]:

fnlwgt	education- num	capital- gain	capital- loss	hours- per- week	workClass_ Federal- gov	workClass_ Local-gov	workClass_ Private	workClass_ Self-emp- inc	...	native- country_ Scotland	native- country_ South	native- country_ Taiwan	native- country_ Thailand	native-country_ Trinidad&Tobago
77516	13	2174	0	40	0	0	0	0	...	0	0	0	0	0
83311	13	0	0	13	0	0	0	0	...	0	0	0	0	0
215646	9	0	0	40	0	0	1	0	...	0	0	0	0	0
234721	7	0	0	40	0	0	1	0	...	0	0	0	0	0
338409	13	0	0	40	0	0	1	0	...	0	0	0	0	0

× 106 columns



Machine Learning – EDA - Plots

EDA - Plots

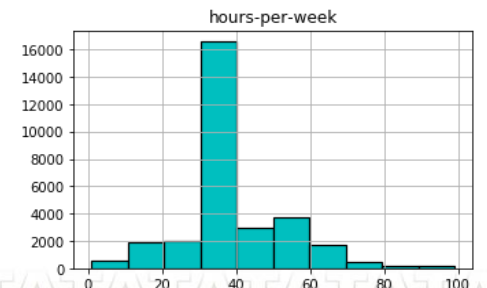
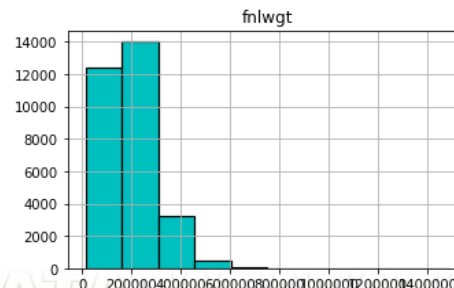
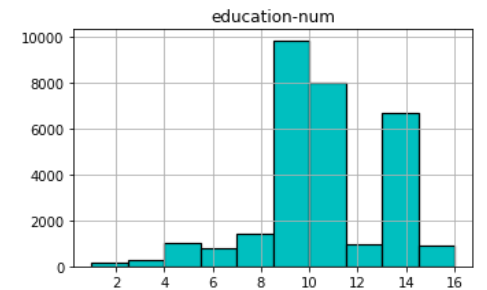
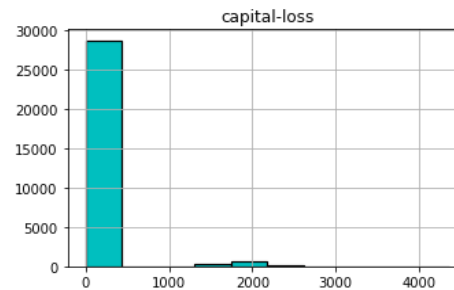
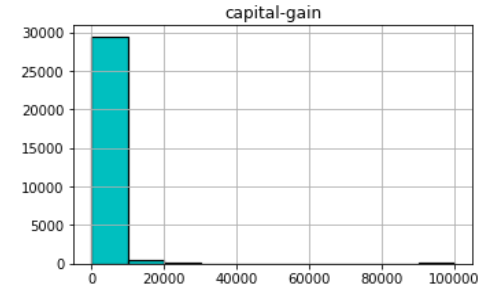
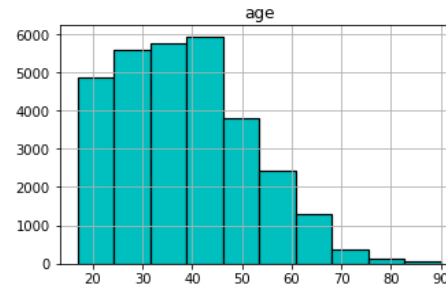
- Exploratory Data Analysis(EDA) is majorly done using plots.
- Some of the important plots...
 - ❖ Histograms
 - ❖ Heat map / Correllogram
 - ❖ Pairs plot

Machine Learning – EDA - Plots

EDA - Plots - Histogram

```
distribution = census_income.hist  
(edgecolor = 'black', linewidth = 1.2,  
color = 'c')
```

```
fig = plt.gcf()  
fig.set_size_inches(12,12)  
plt.show()
```



Machine Learning – EDA - Plots

EDA - Plots – Heat Map

```
plt.figure(figsize=(12,10), dpi= 80)
sns.heatmap(census_income.corr(),xticklabels=census_income.corr().columns,
            yticklabels=census_income.corr().columns, cmap='RdYlGn', center=0, annot=True)
plt.title('Correlogram of Census Income Data', fontsize=24)
plt.xticks(fontsize=14,rotation=90)
plt.show()
```

Machine Learning – EDA - Plots

EDA - Plots

Correlogram of Census Income Data



Machine Learning – EDA - Plots

EDA - Plots – Pairs plot

```
plt.figure(figsize=(10,8), dpi= 80)
sns.pairplot(census_income,kind="scatter",hue="income",plot_kws=dict(s=80,edgecolor="white", linewidth=2.5))
plt.show()
```

Machine Learning – EDA - Plots

EDA - Plots



Thank You